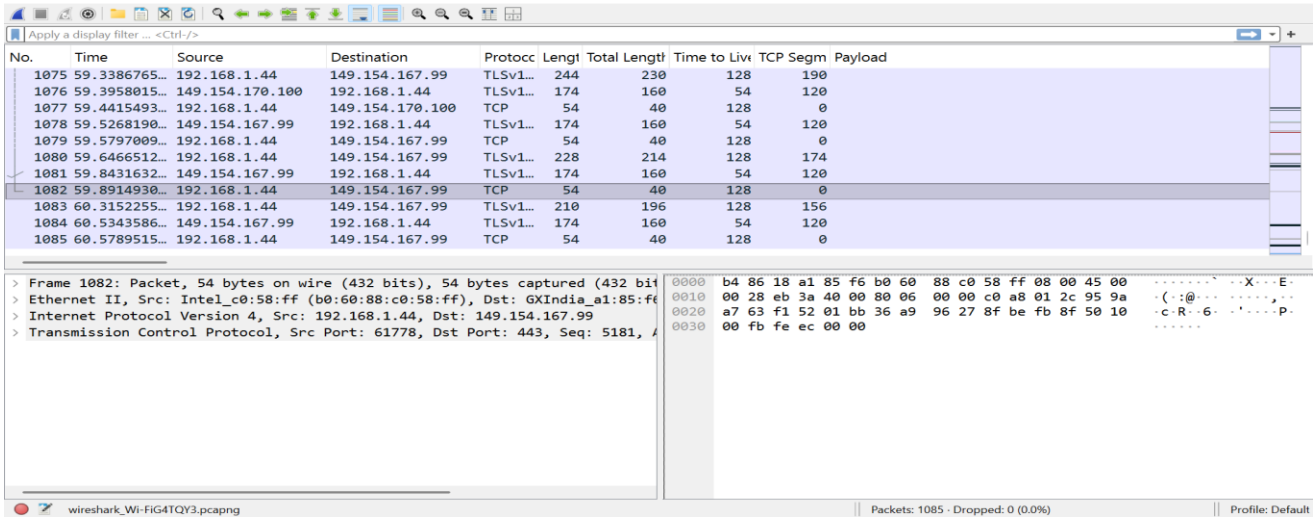
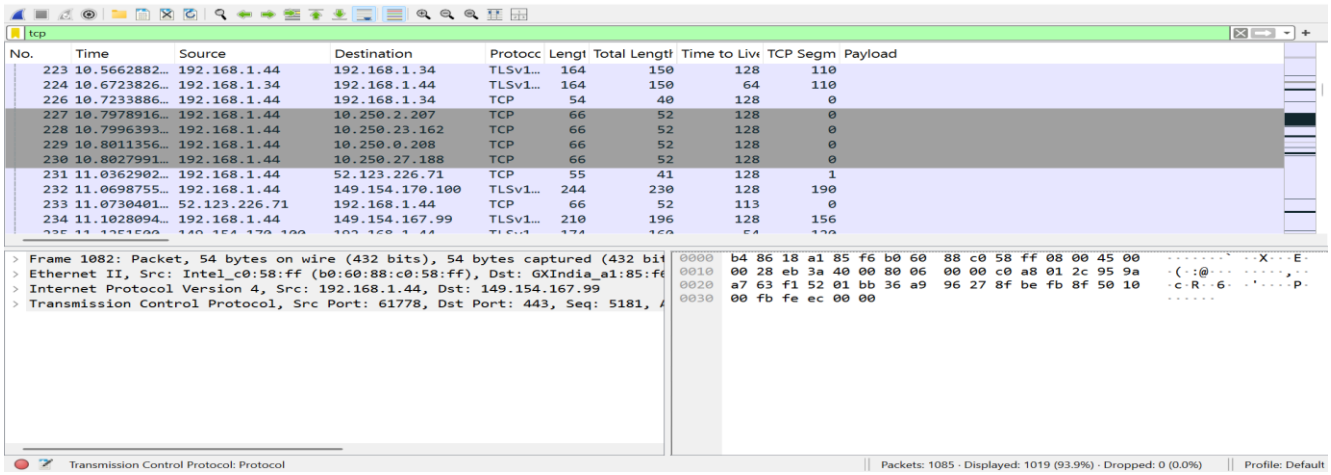
Skill-1

Encapsulation Builder

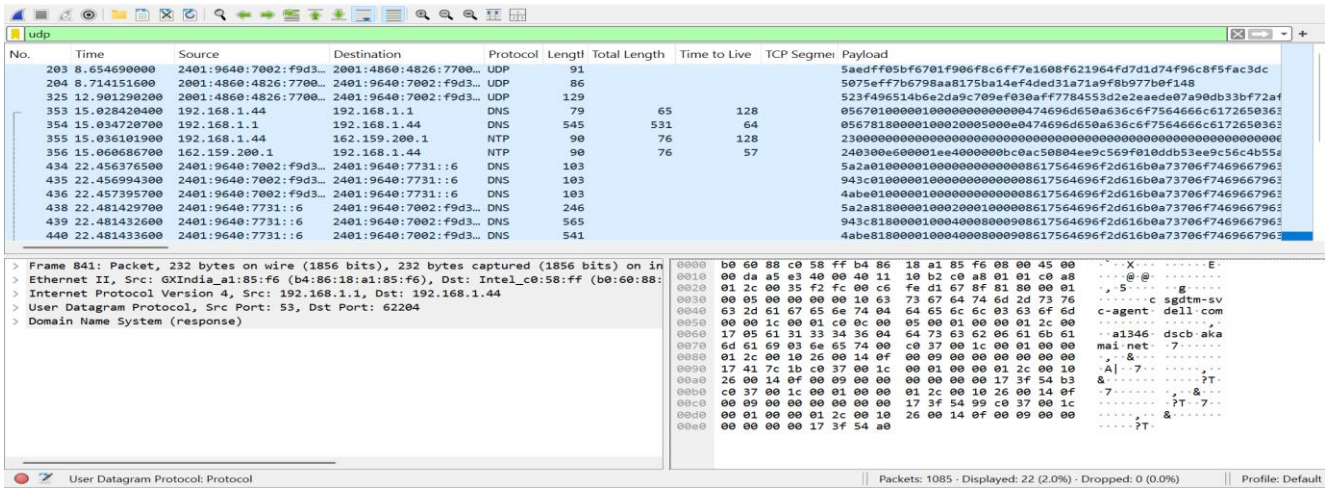
Step 1 to 3:



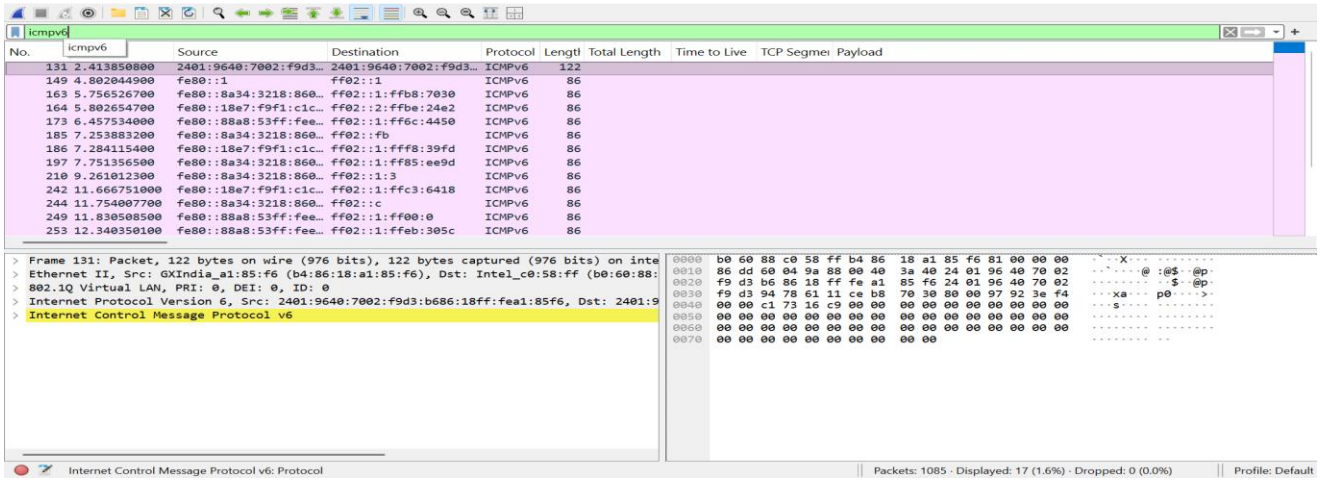
Step 4: tcp



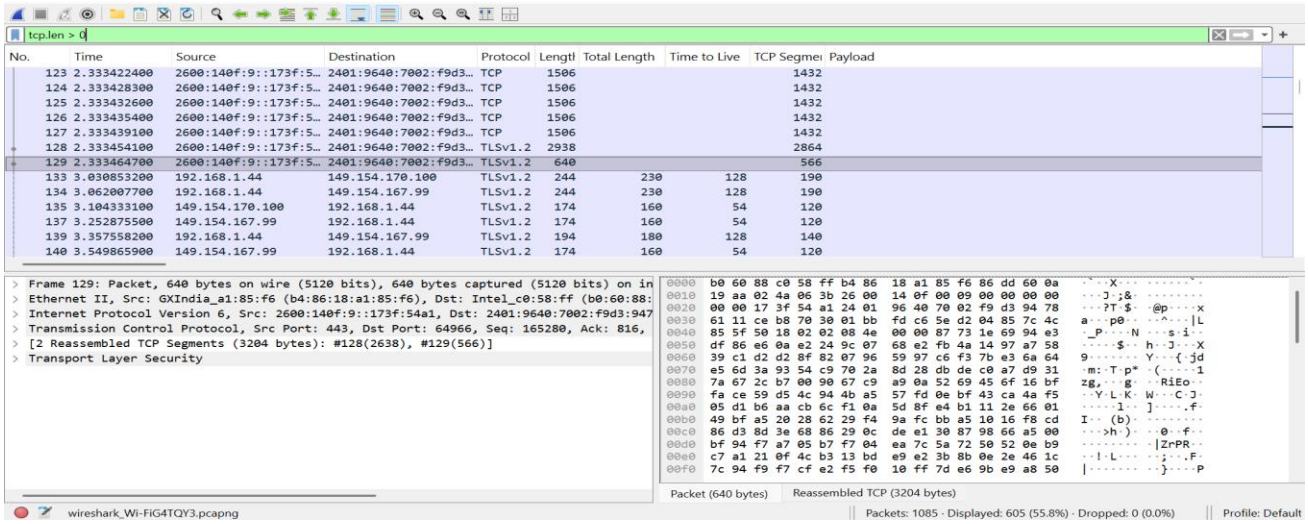
Udp



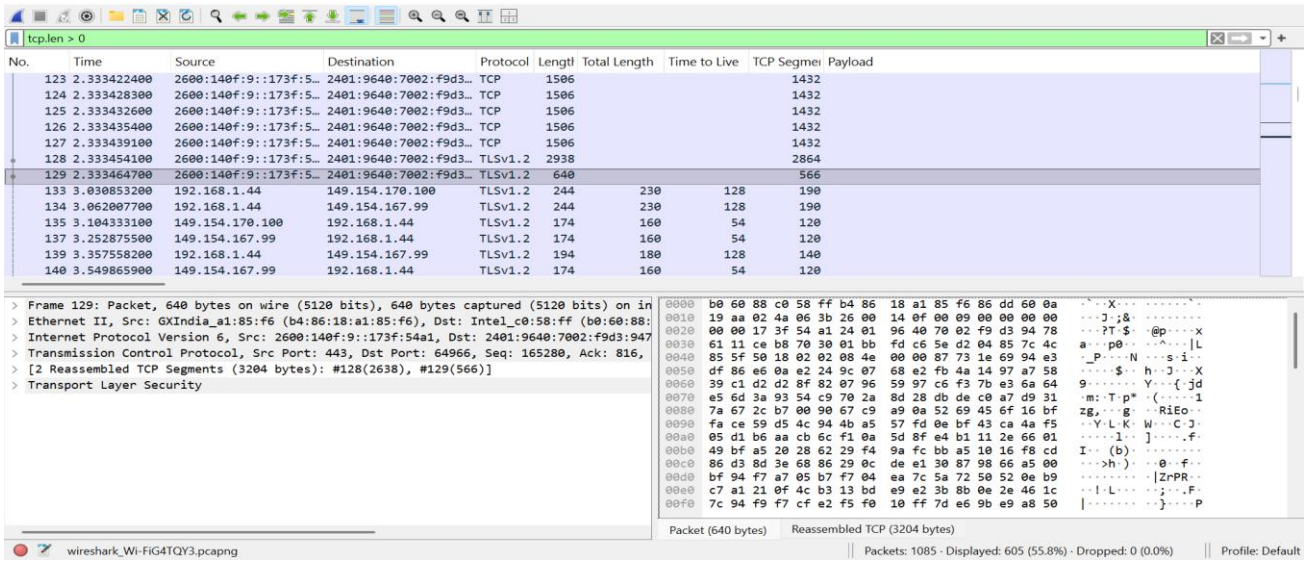
Icmp



tcp.len > 0



Step 5: Inspect packet headers



Parameter	Value	Where to find it
Frame length	244 bytes	Frame section, first line
Ethernet header	14 bytes	Ethernet II
Source MAC	read from screen	Ethernet II, “Src:”
Destination MAC	read from screen	Ethernet II, “Dst:”
IPv4 header length	20 bytes	Internet Protocol Version 4
IPv4 total length	230 bytes	Internet Protocol Version 4
TCP header length	20 bytes	Transmission Control Protocol
TCP segment length	190 bytes	“TCP Segment Len”

Step 6: Calculate header overhead

Part 1: Standard example (from the lab)

Ethernet header = 14 bytes

IPv4 header = 20 bytes

TCP header = 20 bytes

Application data = 1500 bytes

FCS = 4 bytes

Total size = 14 + 20 + 20 + 1500 + 4 = 1558 bytes

Total overhead = 14 + 20 + 20 + 4 = 58 bytes

Payload efficiency = (1500 / 1558) × 100 = 96.28%

Part 2: Your captured packet (packet 133)

Ethernet header = 14 bytes

IPv4 header = 20 bytes

TCP header = 20 bytes

Payload (TCP segment length) = 190 bytes

Frame length = 244 bytes

Check: 14 + 20 + 20 + 190 = 244 □

Total overhead = 14 + 20 + 20 = 54 bytes (Wireshark doesn’t show the FCS)

Payload efficiency = (190 / 244) × 100 = 77.87%

Step 7: Fill in your observation table

Parameter	TCP Packet (No. 1)	UDP Packet (No. 353)
Frame Length	210 bytes	79 bytes
Ethernet Header Length	14 bytes	14 bytes
IP Header Length	20 bytes	20 bytes
Transport Header Length	20 bytes	8 bytes
IP Total Length	196 bytes	65 bytes
Transport Segment/Datagram Length	176 bytes (20 + 156)	45 bytes (8 + 37)
Payload Length	156 bytes	37 bytes

Question and answers

1. What is encapsulation?
Encapsulation is the process where each layer of the network stack adds its own header (and sometimes a trailer) to the data from the layer above before passing it down. Application data becomes a TCP segment or UDP datagram, then an IP packet, then an Ethernet frame. The receiving device reverses this (decapsulation), removing one header at each layer.

2. What is the purpose of the Ethernet header?
The Ethernet header (14 bytes) handles delivery within a local network. It contains the destination MAC address (6 bytes), the source MAC address (6 bytes), and the EtherType (2 bytes), which says which protocol is inside, such as IPv4 or IPv6. In your capture, the source MAC is your laptop (b0:60:88:c0:58:ff) and the destination is your router (b4:86:18:a1:85:f6).

3. What information is stored in the IP header?
The IPv4 header (usually 20 bytes) holds the version, header length, total length, identification, flags and fragment offset, time to live (TTL), protocol number (6 for TCP, 17 for UDP), header checksum, and the source and destination IP addresses. It handles routing across networks. In your capture, the IP addresses are 192.168.1.44 and 149.154.167.99.

4. What is the difference between TCP and UDP headers?

	TCP	UDP
Header size	20 bytes minimum (more with options)	8 bytes fixed
Fields	Source/destination ports, sequence number, acknowledgment number, flags, window size, checksum	Source/destination ports, length, checksum

	TCP	UDP
Connection	Connection-oriented	Connectionless
Reliability	Reliable, with acknowledgments, retransmission, and ordering	Unreliable, with no guarantees
Speed	Slower because of overhead	Faster and lighter
Examples	HTTPS, email, file transfer	DNS, streaming, gaming

5. Why does each protocol add a header?
Each layer has its own job, and it needs control information to do it. The transport layer needs ports to deliver data to the right application, plus sequencing and reliability information. The network layer needs IP addresses to route packets between networks. The data link layer needs MAC addresses to deliver frames on the local link. Headers let each layer work independently and let the receiving side know how to process the data.

6. What is packet overhead?
Packet overhead is the extra bytes in a packet that are not application data, namely the protocol headers (and trailer such as the FCS). For an Ethernet + IPv4 + TCP packet, overhead is 14 + 20 + 20 = 54 bytes (58 with the FCS). In your TCP packet, 54 of the 210 bytes were overhead and 156 were payload.

7. How does header overhead affect network efficiency?
Overhead uses bandwidth that could carry useful data. Efficiency is payload ÷ total frame size × 100. Because header size is fixed, small payloads have low efficiency and large payloads have high efficiency. In the lab's example, a 1500-byte payload gives 96.28%. In your capture, the TCP packet with 156 bytes of payload gives 74.29%, and the UDP packet with only 37 bytes gives 46.84%. So larger packets use the network more efficiently, and tiny packets (like DNS queries or acknowledgments) waste a larger share on headers.

Since these are lab answers your faculty will read, put them in your own phrasing and include your own packet numbers where I've used them. Also make sure the packet numbers in your answers match the ones in your Step 5-7 screenshots and table.